

hw5.ipynb

October 6, 2026

0.1 Part 1

```
[1]: # importing the libraries needed
import pandas as pd
import plotly.express as px
import numpy as np
import requests as rq
import bs4
from io import StringIO

[2]: # reading the webpage data
url = 'https://en.wikipedia.org/w/index.php?
↳title=List_of_countries_by_GDP_(nominal)&oldid=1187446467'
page = rq.get(url, headers={"User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS_
↳X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/17.4 Safari/605.
↳1.15"})
page.text[0:99]
# parsing the webpage
soup = bs4.BeautifulSoup(page.text, 'html.parser')

[3]: # reading the webpage data and extracting tables
tables = pd.read_html(StringIO(str(soup)))
# for loop to print columns
for i in range(len(tables)):
    df = tables[i]
    print(df.columns)
```

```
Index([0], dtype='int64')
Index([0, 1, 2], dtype='int64')
MultiIndex([( 'Country/Territory', 'Country/Territory'),
            ( 'UN region', 'UN region'),
            ( 'IMF[1][13]', 'Forecast'),
            ( 'IMF[1][13]', 'Year'),
            ( 'World Bank[14]', 'Estimate'),
            ( 'World Bank[14]', 'Year'),
            ( 'United Nations[15]', 'Estimate'),
            ( 'United Nations[15]', 'Year')],
          )
Index(['vteLists of countries by financial rankings', 'vteLists of countries by
```

```
financial_rankings.1'], dtype='str')
Index(['vteLists of countries by GDP rankings', 'vteLists of countries by GDP
rankings.1'], dtype='str')
Index(['vteEconomic classification of countries', 'vteEconomic classification of
countries.1'], dtype='str')
Index([0, 1], dtype='int64')
```

```
[4]: # selecting the third table
df = tables[2]
df.head()
```

```
[4]: Country/Territory UN region IMF[1][13] World Bank[14] \
      Country/Territory UN region Forecast Year Estimate Year
0 World - 104476432 2023 100562011 2022
1 United States Americas 26949643 2023 25462700 2022
2 China Asia 17700899 [n 1]2023 17963171 [n 3]2022
3 Germany Europe 4429838 2023 4072192 2022
4 Japan Asia 4230862 2023 4231141 2022

      United Nations[15]
      Estimate Year
0 96698005 2021
1 23315081 2021
2 17734131 [n 1]2021
3 4259935 2021
4 4940878 2021
```

```
[5]: # selecting the columns we need
clean_df = df.iloc[:, [0, 1, 2, 4, 6]].copy()
clean_df.columns = ['Country', 'Region', 'IMF', 'WorldBank', 'UN']
clean_df.head()
```

```
[5]: Country Region IMF WorldBank UN
0 World - 104476432 100562011 96698005
1 United States Americas 26949643 25462700 23315081
2 China Asia 17700899 17963171 17734131
3 Germany Europe 4429838 4072192 4259935
4 Japan Asia 4230862 4231141 4940878
```

```
[6]: # types for the data
clean_df.dtypes
```

```
[6]: Country      str
      Region      str
      IMF        str
      WorldBank  str
      UN         str
      dtype: object
```

```
[7]: # converting to numeric values
for col in ['IMF', 'WorldBank', 'UN']:
    clean_df[col] = (
        clean_df[col]
        .astype(str)
        .str.replace(',', '', regex=False)
        .str.strip()
    )
    clean_df[col] = pd.to_numeric(clean_df[col], errors='coerce')

clean_df.head()
```

```
[7]:
```

	Country	Region	IMF	WorldBank	UN
0	World	-	104476432.0	100562011.0	96698005.0
1	United States	Americas	26949643.0	25462700.0	23315081.0
2	China	Asia	17700899.0	17963171.0	17734131.0
3	Germany	Europe	4429838.0	4072192.0	4259935.0
4	Japan	Asia	4230862.0	4231141.0	4940878.0

```
[8]: # checking for missing values
print("\nMissing values")
print(clean_df[['IMF', 'WorldBank', 'UN']].isna().sum())
```

```
Missing values
IMF          22
WorldBank    27
UN           1
dtype: int64
```

```
[9]: # remove the world row
clean_df = clean_df[clean_df['Country'] != 'World'].copy()

# fills in the missing data from Worldbank and UN
clean_df['IMF_imputed'] = clean_df['IMF'].fillna(clean_df['WorldBank']).
    ↪fillna(clean_df['UN'])

# check to make sure no more missing vals
print("Missing values in IMF_imputed:", clean_df['IMF_imputed'].isna().sum())
clean_df.head()
```

```
Missing values in IMF_imputed: 0
```

```
[9]:
```

	Country	Region	IMF	WorldBank	UN	IMF_imputed
1	United States	Americas	26949643.0	25462700.0	23315081.0	26949643.0
2	China	Asia	17700899.0	17963171.0	17734131.0	17700899.0
3	Germany	Europe	4429838.0	4072192.0	4259935.0	4429838.0
4	Japan	Asia	4230862.0	4231141.0	4940878.0	4230862.0

5	India	Asia	3732224.0	3385090.0	3201471.0	3732224.0
---	-------	------	-----------	-----------	-----------	-----------

```
[10]: # create the stacked plot
fig = px.bar(clean_df, x = "Region", y = "IMF", color = "Country")
fig.show() # display the html graphic

# save as html
fig.write_html("stacked_bar.html")
```

0.2 Part 2

```
[11]: import plotly.graph_objects as go
# load in data
lookup_url = "https://raw.githubusercontent.com/bcaffo/MRCloudT1volumetrics/
↳master/inst/extdata/multilevel_lookup_table.txt"
multilevel_lookup = pd.read_csv(lookup_url, sep="\t").drop(['Level5'], axis=1)
multilevel_lookup = multilevel_lookup.rename(columns={
    "modify": "roi",
    "modify.1": "level4",
    "modify.2": "level3",
    "modify.3": "level2",
    "modify.4": "level1"
})
multilevel_lookup = multilevel_lookup[['roi', 'level4', 'level3', 'level2',
↳'level1']]

# load in the data for that subject
data_url = "https://raw.githubusercontent.com/smart-stats/ds4bio_book/main/book/
↳assets/kirby21AllLevels.csv"
subjectData = pd.read_csv(data_url)
subjectData = subjectData.loc[(subjectData.type == 1) & (subjectData.level ==
↳5) & (subjectData.id == 127)]
subjectData = subjectData[['roi', 'volume']]

# merge the two tables
subjectData = pd.merge(subjectData, multilevel_lookup, on="roi")
subjectData = subjectData.assign(icv="ICV")
subjectData = subjectData.assign(comp=subjectData.volume / np.sum(subjectData.
↳volume))

subjectData.head()
```

```
[11]:
```

	roi	volume	level4	level3	level2	level1	\
0	SFG_L	12926	SFG_L	Frontal_L	CerebralCortex_L	Telencephalon_L	
1	SFG_R	10050	SFG_R	Frontal_R	CerebralCortex_R	Telencephalon_R	
2	SFG_PFC_L	12783	SFG_L	Frontal_L	CerebralCortex_L	Telencephalon_L	
3	SFG_PFC_R	11507	SFG_R	Frontal_R	CerebralCortex_R	Telencephalon_R	

```

4 SFG_pole_L    3078 SFG_L Frontal_L CerebralCortex_L Telencephalon_L

    icv      comp
0 ICV  0.009350
1 ICV  0.007270
2 ICV  0.009247
3 ICV  0.008324
4 ICV  0.002227

```

```

[12]: # define the transitions of my sankey diagram
transitions = [
    ('icv', 'level1'),
    ('level1', 'level2'),
    ('level2', 'level3')
]

# create lists of dataframes for each transition
link_dfs = []
for src_col, tgt_col in transitions:
    temp = subjectData.groupby([src_col, tgt_col])['volume'].sum().reset_index()
    temp.columns = ['source_label', 'target_label', 'volume']
    link_dfs.append(temp)

# make it all one dataframe
links_df = pd.concat(link_dfs, ignore_index=True)
links_df.head(10)

```

```

[12]:  source_label  target_label  volume
0         ICV          CSF    109787
1         ICV  Diencephalon_L    11817
2         ICV  Diencephalon_R    11560
3         ICV   Mesencephalon    10271
4         ICV   Metencephalon    159411
5         ICV  Myelencephalon     4975
6         ICV  Telencephalon_L    531153
7         ICV  Telencephalon_R    543444
8         CSF        Sulcus_L     33975
9         CSF        Sulcus_R     30018

```

```

[13]: # get the labels
all_labels = pd.concat([links_df['source_label'], links_df['target_label']]).
    .drop_duplicates().tolist()

# get the indices for each label
source_indices = [all_labels.index(name) for name in links_df['source_label']]
target_indices = [all_labels.index(name) for name in links_df['target_label']]
volumes = links_df['volume'].tolist()

```

```

# creating my sankey diagram
fig_sankey = go.Figure(data=[go.Sankey(
    node = {
        'label': all_labels,
        'pad': 15,
        'thickness': 20
    },
    link = {
        'source': source_indices,
        'target': target_indices,
        'value': volumes
    }
)])
# update the sankey layout
fig_sankey.update_layout(
    title_text=" MRI Data Hierarchy",
    font_size=10,
    height=700
)

# display the plot
fig_sankey.show()

# saving as html
fig_sankey.write_html("sanky.html")

```

0.3 Link to sanky diagram

<https://github.com/sramirez44/Assignment-5/blob/main/sanky.html>